Solution Requirements

Date	02 July 2026
Team ID	
Project Name	Darshan Ease – Temple Darshan Ticket Booking System
Maximum Marks	4 Marks

Define Functional and Non-Functional Requirements:

Create a comprehensive list of requirements to define exactly what your solution will do and how well it will perform.

- Functional Requirements (FR) define specific behaviors, functions, or features of the system (What the system should do).
- Non-Functional Requirements (NFR) specify the criteria that judge the operation of a system, rather than specific behaviors (How the system should be).

Step 1: Functional Requirements (FR)

S.No	Requirement Category	Requirement Description	Priority
1	Authentication	Users register and log in with email and password. On successful login the system issues a JWT (JSON Web Token), which is validated on every subsequent request to verify the user's identity.	High
2	Authorization levels	Two roles are supported: Devotee (browse temples, book/view/cancel own darshan slots) and Admin (manage temples, darshan slots, and view all bookings).	High
3	External interfaces	No third-party payment gateway is used in the current phase. The React frontend communicates with the Express backend through internal REST APIs, backed by a MongoDB database.	Medium
4	Transactions processing	Order requests are validated and stored in the Orders collection. The selected product and user details are linked successfully.	High
5	Reporting	Admin/User can view product details, orders, and booking history.	Medium
6	Business rules, etc.	Every order must be linked to a valid registered user and an existing product	High
7	Compliance to laws or regulations	User passwords are stored securely and personal data is used only for booking purposes. No specific regulatory certification (e.g., GDPR/HIPAA) is targeted for this academic-scope project.	Low
8	Other	Users can view their order history from the Orders page	Medium

Step 2: Non-Functional Requirements (NFR)

S.No	NFR Category	Requirement Description	Target Metric / Acceptance Criteria
1	Performance & Speed	Booking, login, and Login, Products, Orders, and Profile APIs should respond within 3 seconds.	Page loads and API responses in < 3 seconds.
2	Scalability	The Node.js/Express backend with MongoDB is designed to scale horizontally as more devotees and temples are added.	Supports up to 500 concurrent users for the current academic scope.
3	Security & Data Privacy	Passwords are hashed before storage, and protected routes require a valid JWT to access booking or profile data.	Passwords hashed using bcrypt; protected APIs reject requests without a valid token.
4	Reliability & Availability	The backend server is designed to recover automatically during development/testing so bookings are not lost.	99% uptime during the evaluation/demo period.
5	Usability & Accessibility	A simple, responsive React interface provides clear navigation across home, Home, Products, Login, Register, Orders, and Profile pages.	Usable on both desktop and mobile browsers.
6	Other	Maintainability: the backend follows the MVC pattern (models, controllers, routes), making it easier to add new features later.	Clear separation of models, controllers, and routes across the codebase.